

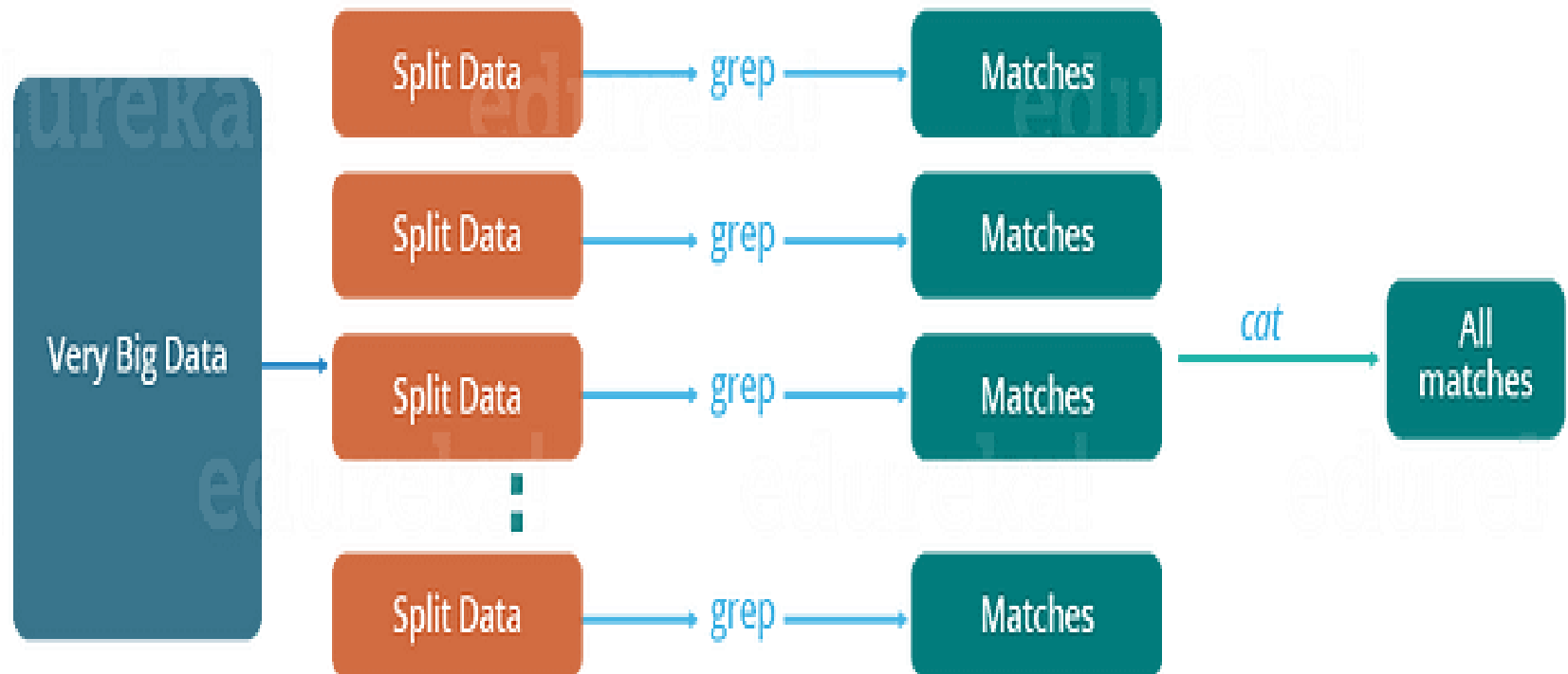
Background

- Traditional programming is serial
- Parallel programming
 - Break processing into parts that can be executed concurrently on multiple processors
- Challenge
 - Identify tasks that can run concurrently
 - and/or groups of data that can be processed concurrently
 - Not all problems can be parallelized

Motivation

edureka!

The Traditional Way



Big Data

```
graph TD; A[Big Data] --> B["• it cannot be stored in one machine"]; A --> C["• it cannot be processed in one machine"]; B --> D["• store the data sets on multiple machines"]; D --> E["• Google File System"]; C --> F["• parallelize computation on multiple machines"]; F --> G["• MapReduce"]; E --- G; H[Today!] --- G;
```

- it cannot be **stored** in one machine



- store the data sets on multiple machines



- **Google File System**

- it cannot be **processed** in one machine



- parallelize computation on multiple machines



Today!

- **MapReduce**

Motivation: Large Scale Data Processing

- Want to process lots of data (> 1 TB)
- Want to parallelize across hundreds/thousands of CPUs
- Want to make this easy

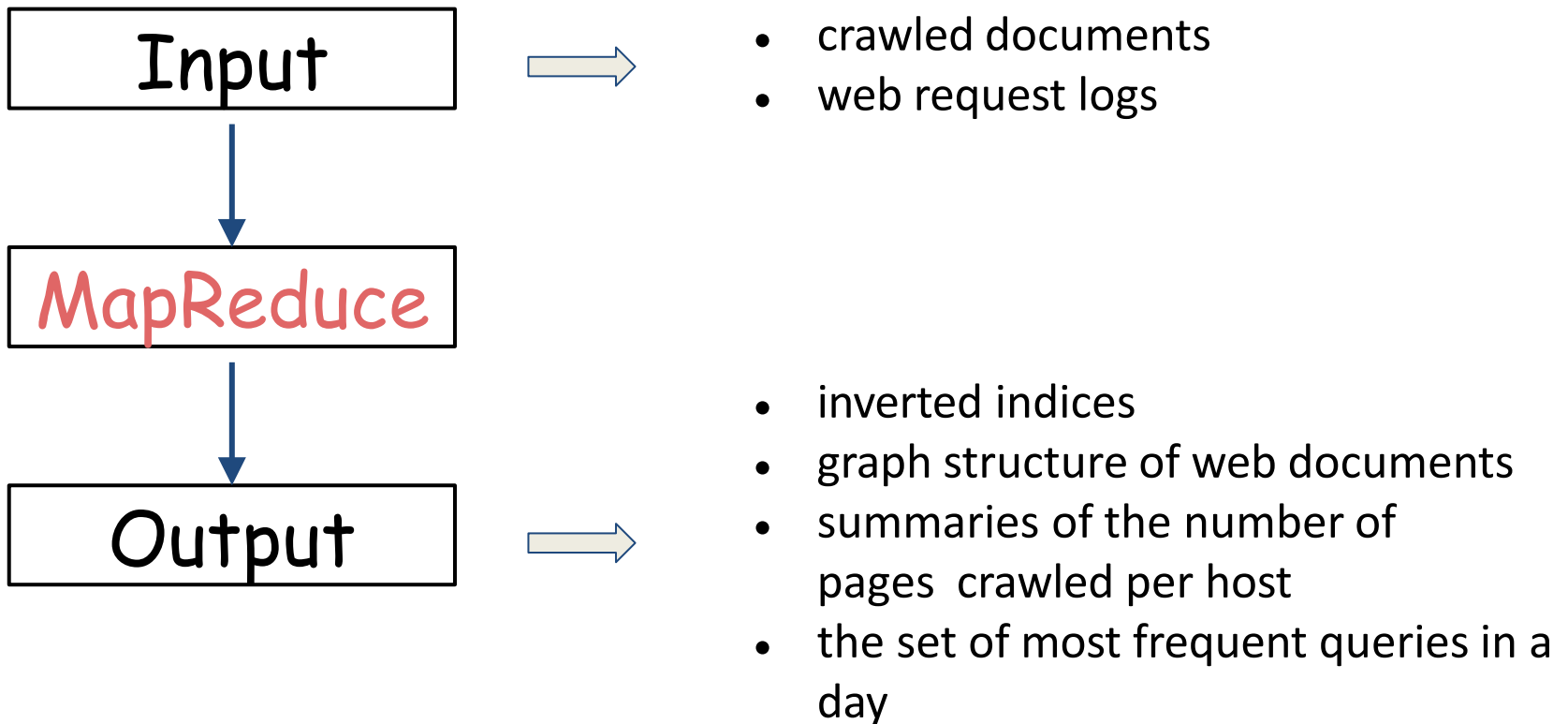
Motivation: Example

- Processing Web Data on a Single Machine
 - 20+ billion web pages x 20KB = 400+ terabytes
 - One computer can read 30-35 MB/sec from disk
 - ~ four months to read the web
 - ~1,000 hard drives just to store the web
 - Even more to do something with the data
- Takes too long on a single machine, but with 1000 machines?
 - < 3 hours to perform on 1000 machines
 - But how long to program? What about the overheads?
 - Communication, coordination, recovery from machine failure
 - Status Reporting, Debugging, Optimization, Locality

The Map-Reduce Framework

- Google published MapReduce 2004, a year after the Google File System
- A high level programming paradigm
 - allows many important data-oriented processes to be written simply
- processes large data by:
 - applying a function to each logical record in the input (**map**)
 - categorize and combine the intermediate results into summary values (**reduce**)

Where does Google use MapReduce?



Storage Model

- Data is stored in large files (TB, PB)
 - e.g., market-basket data (more when we do data mining)
 - or web data
- Files are divided into chunks
 - Typically, many MB (64 MB)
 - sometimes each chunk is replicated for fault tolerance

MapReduce

- **MapReduce**
 - Framework for parallel computing
 - Programmers get simple API
 - Don't have to worry about handling
 - parallelization
 - data distribution
 - load balancing
 - fault tolerance
- Allows to process huge amounts of data (terabytes and petabytes) on thousands of processors

Who has it?

- **Google**
 - Original proprietary implementation
- **Apache Hadoop MapReduce**
 - Most common (open-source) implementation
 - Built to specs defined by Google
- **Amazon Elastic MapReduce**
 - Uses Hadoop MapReduce running on Amazon EC2
 - ... or Microsoft Azure HDInsight
 - ... or Google Cloud MapReduce for App Engine

MapReduce Features

- Automatic parallelization & distribution
- Fault-tolerant
- Provides status and monitoring tools
- Clean abstraction for programmers

MapReduce Working

- Map

- Grab the relevant data from the source
- User function gets called for each chunk of input
- Spits out (key, value) pairs

- Reduce

- Aggregate the results
- User function gets called for each unique key with all values corresponding to that key

MapReduce Working

- Map: (input data) → intermediate(key/value pairs)
- Reduce: intermediate(key/value pairs) → result files

MapReduce Working

- Map

- Grab the relevant data from the source (parse into key, value)
- Write it to an intermediate file

- Partition

- Partitioning: identify which of R reducers will handle which keys
- Map partitions data to target it to one of R Reduce workers based on a partitioning function (both R and partitioning function user defined)

Map Worker

- Shuffle & Sort

- Shuffle: Fetch the relevant partition of the output from all mappers
- Sort by keys (different mappers may have sent data with the same key)

- Reduce

- Input is the sorted output of mappers
- Call the user *Reduce* function per key with the list of values for that key to aggregate the results

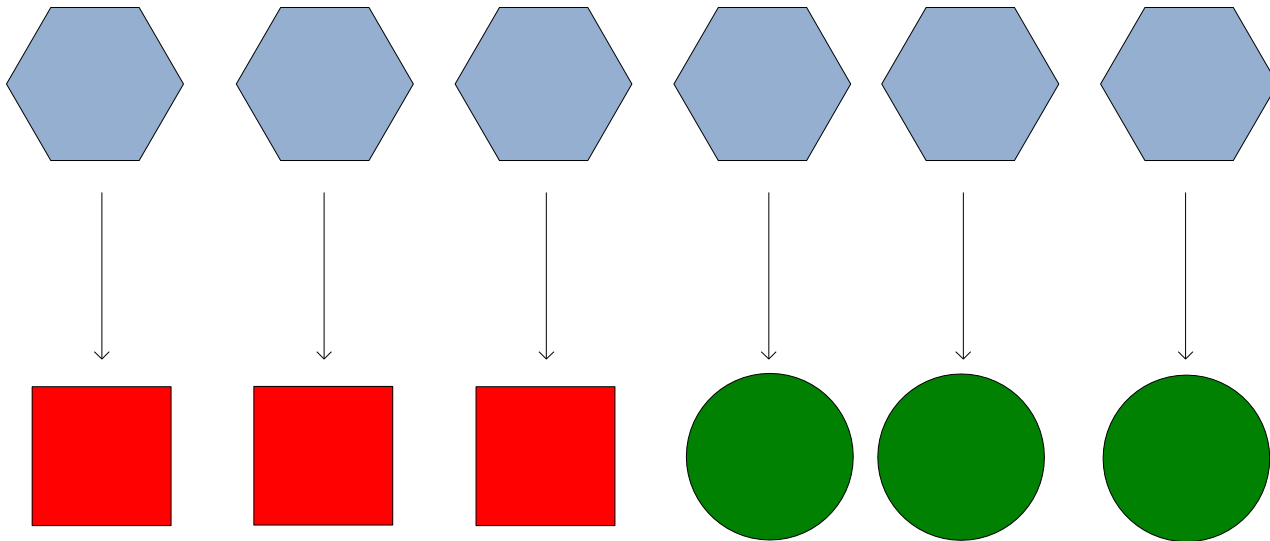
Reduce Worker

Map

- Records from the data source (lines out of files, rows of a database, etc) are fed into the map function as key*value pairs: e.g., (filename, line).
- map() produces one or more intermediate values along with an output key from the input.

Map

```
map (in_key, in_value) ->  
    (out_key, intermediate_value) list
```

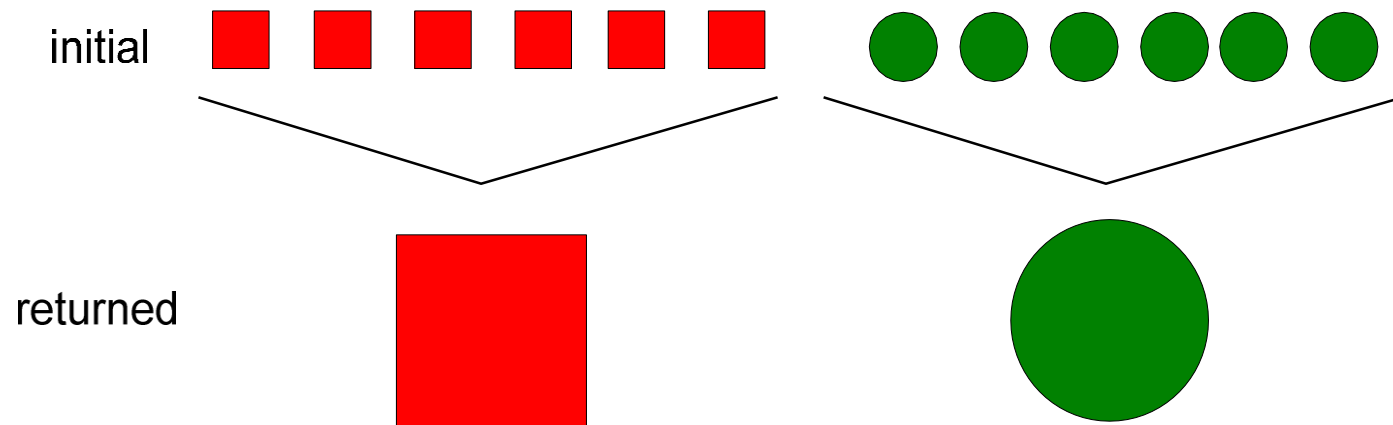


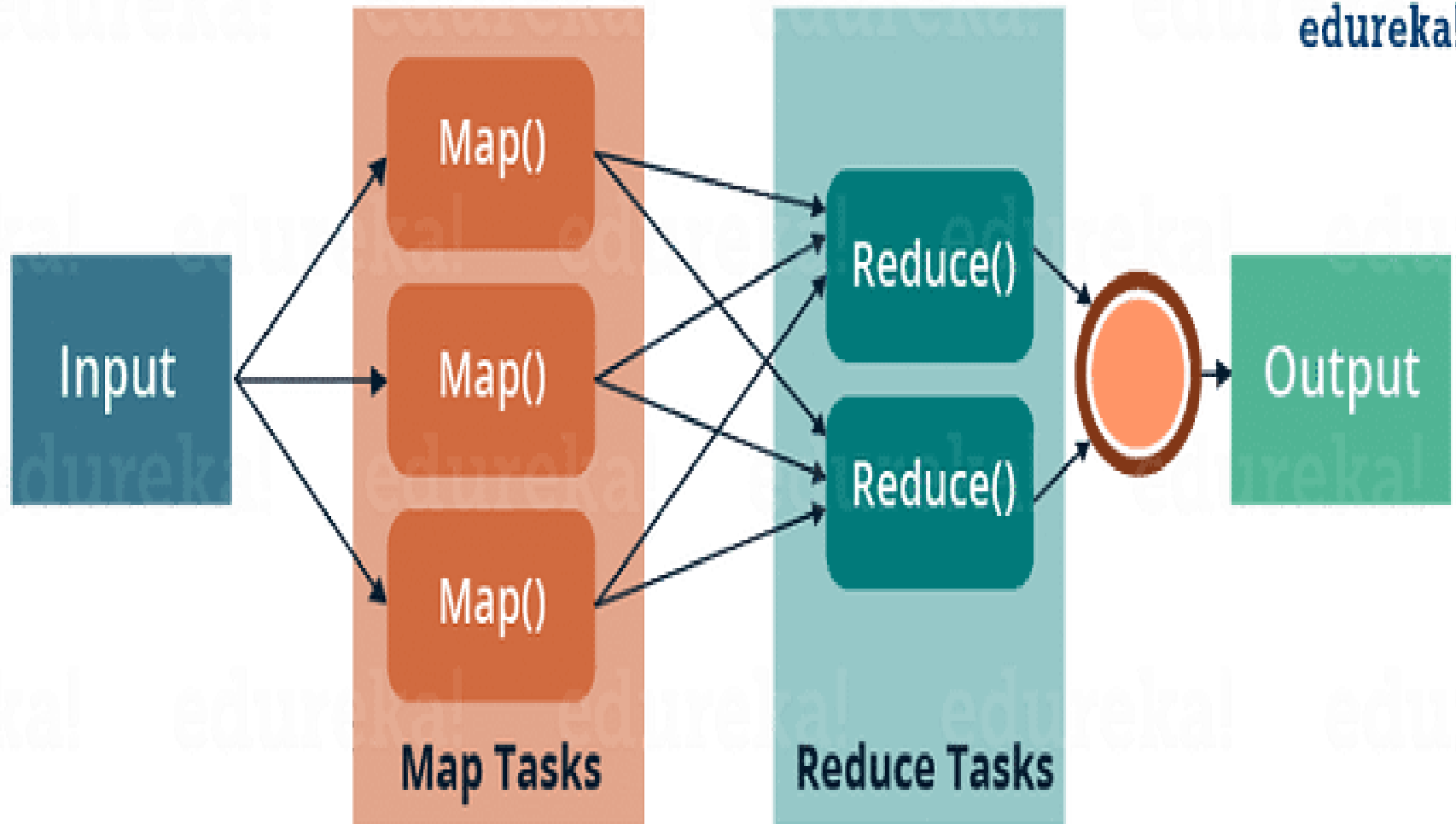
Reduce

- After the *Map* phase is over, all the intermediate values for a given output key are combined into a list
- `reduce()` combines those intermediate values into one or more final values for that same output key
- In practice, usually only one final value per key

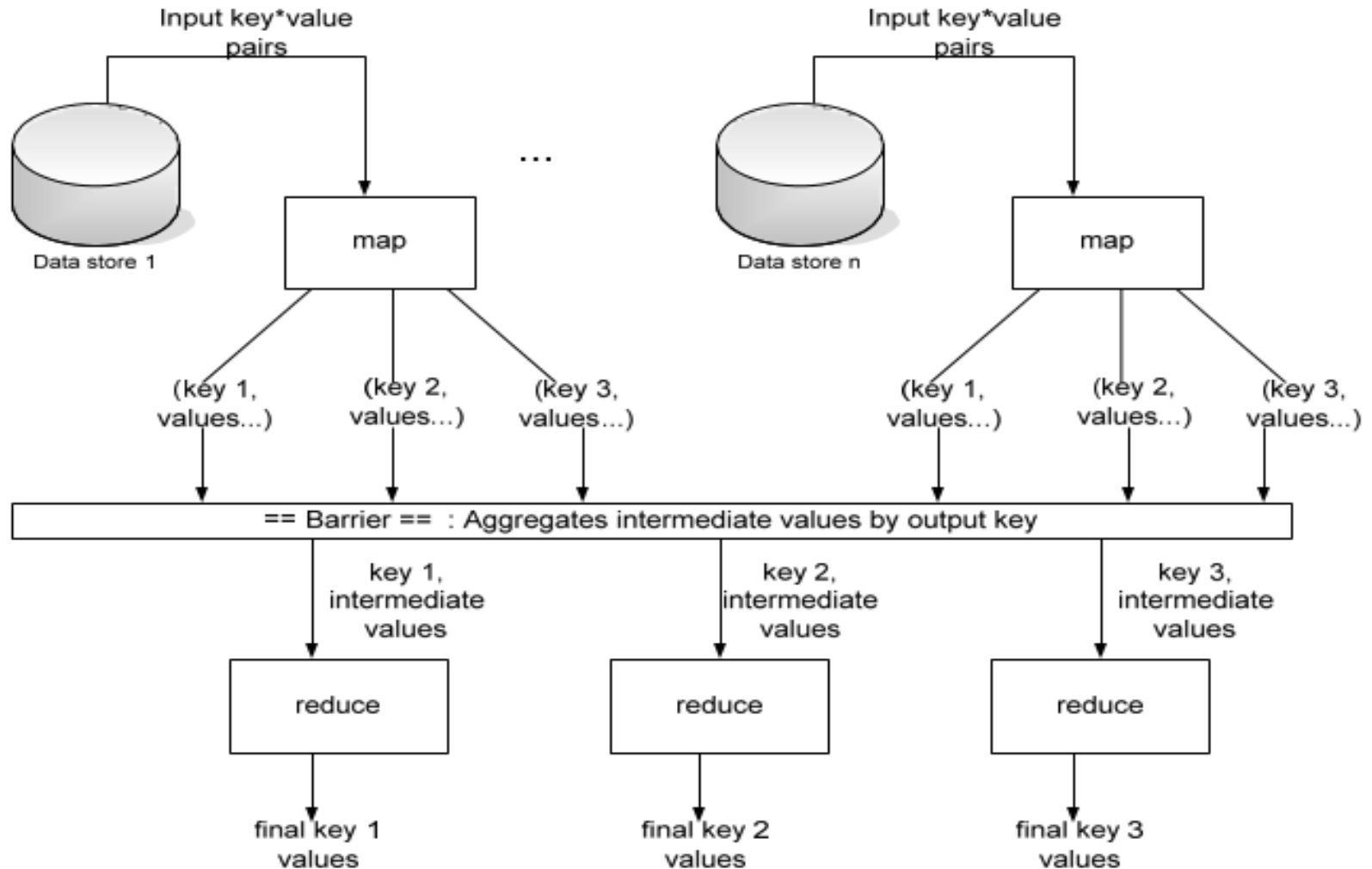
Reduce

```
reduce (out_key, intermediate_value list) -  
      > out_value list
```





MapReduce Working



Parallelism

- `map()` functions run in parallel, creating different intermediate values from different input data sets
 - The programmer must specify a proper input split (chunk) between mappers to enable parallelism
- `reduce()` functions also run in parallel, each working on a different output key
 - Number of reducers is a key parameter which determines MapReduce performance
- All values are processed independently
- Bottleneck: reduce phase can't start until map phase is finished.

Data Locality

- Master program divides up tasks based on location of data
 - Tries to have map() tasks on same machine as physical file data, or at least same rack
- map() task inputs are divided into 64 MB blocks same size as Google File System chunks

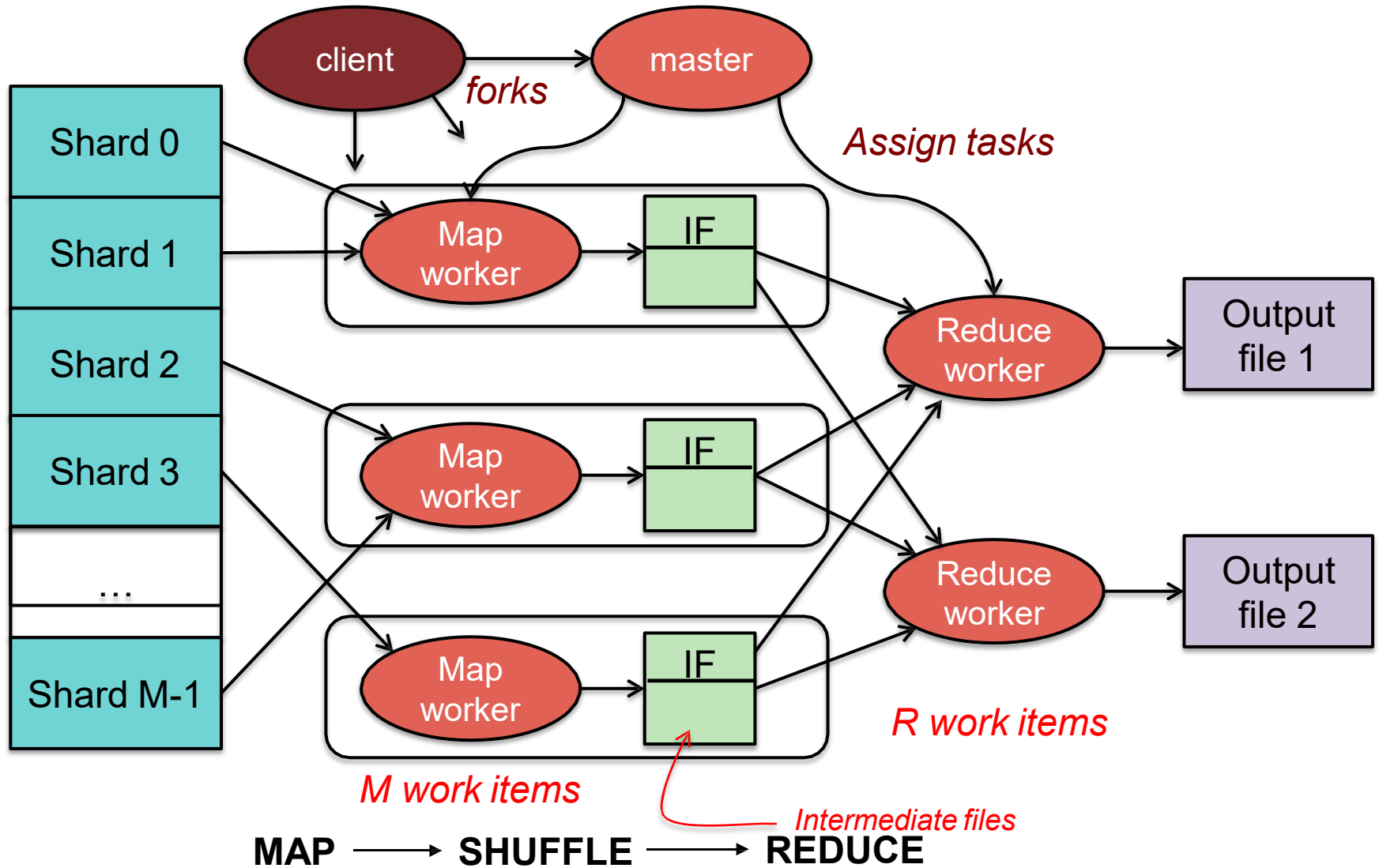
Handling Fault Tolerance

- Although the probability of a machine failure is low, the probability of a machine failing among thousands of machines is common
- Worker Failure
 - The master sends heartbeat to each worker node
 - If a worker node fails, the master reschedules the tasks handled by the worker
- Master Failure
 - The whole MapReduce job gets restarted through a different master

Optimizations

- No reduce can start until map is complete:
 - A single slow disk controller can rate-limit the whole process
- Master redundantly executes “slow-moving” map tasks; uses results of first copy to finish

MapReduce: the complete picture



Why is Map-Reduce Popular?

- Distributed computation before MapReduce
 - how to divide the workload among multiple machines?
 - how to distribute data and program to other machines?
 - how to schedule tasks?
 - what happens if a task fails while running?
 - ... and ... and ...
- Distributed computation after MapReduce
 - how to write Map function?
 - how to write Reduce function?
- Developers' tasks made easy

Example

- Dear, Bear, River, Car, Car, River, Deer, Car and Bear

The Overall MapReduce Word Count Process

edureka!

